

Autor(es):

Wilner Manzanares, CI: 31.654.161
Wilfredo García, CI: 32.097.104
Dannielys Sandoval, CI: 32.424.126
Rodrigo Moreno, CI: 31.802.095

Profesor(a) Guía:
Licda. Ivonne Medina

Tutor(a):
Jhinel Arcaya

Manual de Usuario: EduBot Forest Studio

Bienvenido a la guía oficial de los bloques de programación visual de EduBot. Esta herramienta escanea tu lienzo visual y traduce cada bloque a la sintaxis exacta de Python, cuidando los espacios en blanco (indentación) que Python exige para funcionar.

A continuación, explicamos cada bloque disponible y cómo se combinan para crear algoritmos reales.

Bloques Básicos (Entrada, Salida y Datos)

Estos son los cimientos de cualquier programa. Te permiten interactuar con el usuario y guardar información.

- **Variable:** Actúa como una "caja vacía" para guardar datos (números, texto, resultados) y usarlos más adelante.
- **Input:** Pide información al usuario a través de la terminal.
- **Print:** Muestra mensajes o el valor de las variables terminadas en la terminal.
- **Math:** Realiza operaciones matemáticas básicas (+, -, *, /).

Ejemplo Práctico: Validador de Edad

Este ejemplo pide datos, hace un cálculo y toma una decisión de forma secuencial.

1. Usa **Input** para pedir el "año actual" y el "año de nacimiento", y guárdalos en una variable.

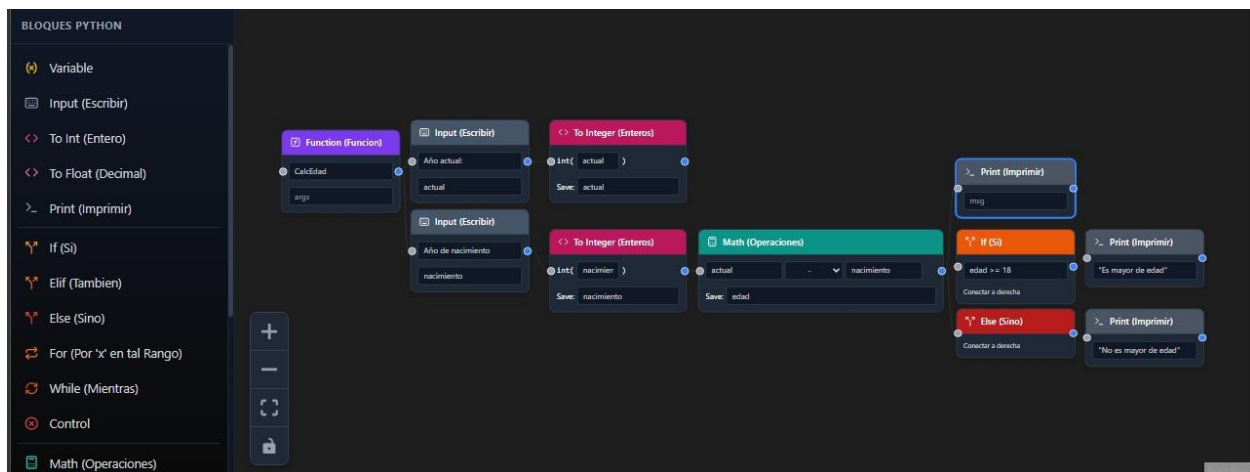
2. Usa **To Int** para cambiar el texto guardado en la variable a un valor numérico.
3. Usa **Math** para restar ambos años y guárdalo en una variable llamada edad.
4. Con un **Print**, muestra la edad calculada.
5. Usa **If y Else** (≥ 18) para imprimir si es mayor de edad o de lo contrario imprimir que es menor de edad.

Así se ve la traducción de tus bloques en EduBot:

```
anio_actual = int(input("Ingresa el año actual: "))
anio_nacimiento = int(input("Ingresa tu año de nacimiento: "))
```

```
edad = anio_actual - anio_nacimiento
print(edad)
```

```
if edad >= 18:
    print("Eres mayor de edad.")
else:
    print("Eres menor de edad.")
```



Bloques de Control de Flujo

Estos bloques permiten que tu programa tome decisiones o repita tareas, alterando la secuencia de arriba hacia abajo.

- **If:** Evalúa una condición. Si es verdadera, ejecuta el código en su interior; si es falsa, lo ignora o ejecuta un camino alternativo.
- **Compare:** Compara dos valores ($>$, $<$, $==$). Es el compañero indispensable del bloque If.

- **For:** Repite una acción un número específico de veces (ej. `range(10)`).
- **While:** Repite una acción infinitamente mientras una condición siga siendo verdadera.
- **Control:** Modifica el comportamiento de los bucles usando `Break` (llave de salida de emergencia), `Continue` (salta un turno) o `Pass` (no hace nada).

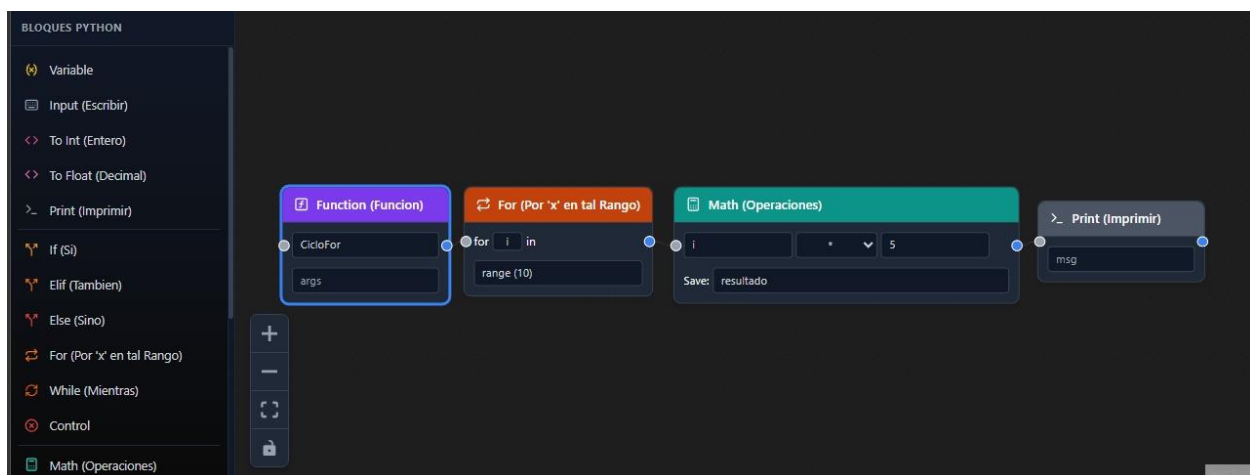
Ejemplo Práctico: Bucle For (Tabla de Multiplicar)

1. Coloca un bloque **For** configurado con `range(10)`.
2. Dentro, usa **Math** para multiplicar la variable del bucle por 5.
3. Usa **Print** para mostrar el resultado en cada vuelta.

Traducción de EduBot (Tabla del 5)

```
print("Tabla del 5:")
```

```
for i in range(10):
    resultado = i * 5
    print(resultado)
```



Ejemplo Práctico: Bucle While (Cuenta Regresiva)

1. Crea una **Variable** cuenta con el valor 5.
2. Usa un **While** con la condición `cuenta > 0`.
3. Dentro, imprime la cuenta y usa **Math** para restar 1 a la variable.
4. Al salir del bucle, usa un **Print** abajo del **Ciclo For** que diga "¡Despegue!".

Traducción de EduBot (Cuenta regresiva)

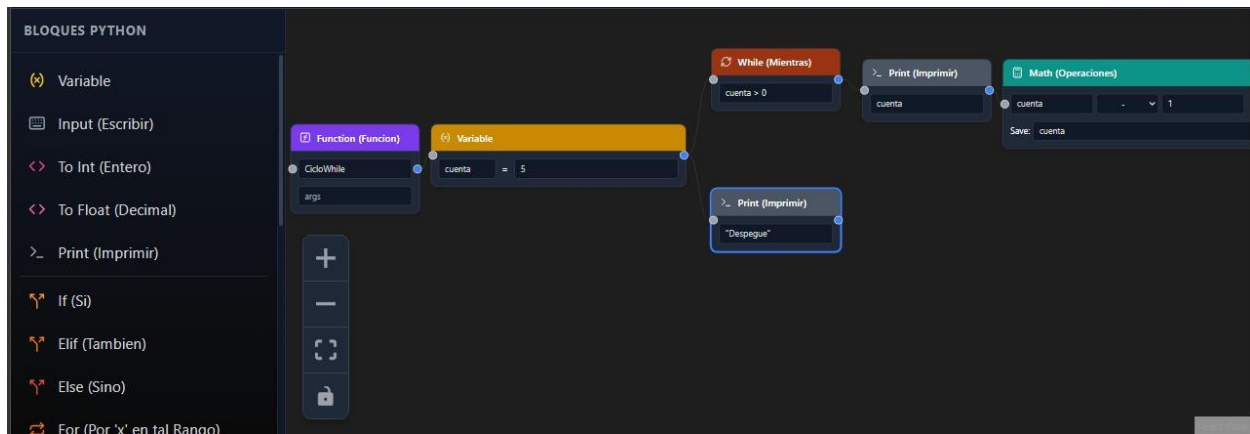
```
import time # Añadimos time para el efecto dramático en Jupyter
```

```

cuenta = 5
while cuenta > 0:
    print(cuenta)
    cuenta = cuenta - 1
    time.sleep(1) # Pausa de 1 segundo

print("¡Despegue!")

```



Bloques Avanzados y Estructuras

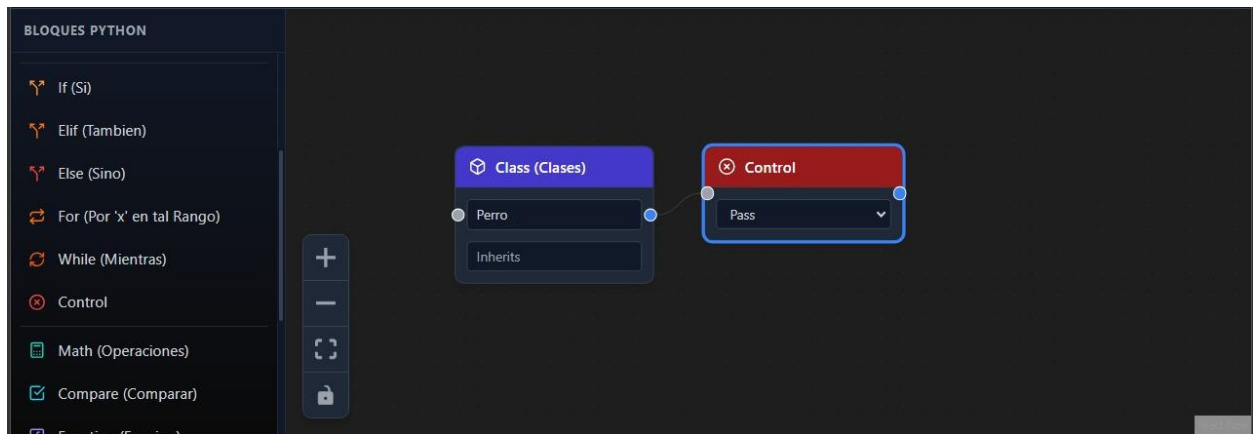
Para programas más complejos, necesitas organizar tu código y protegerlo de errores.

- **Function:** Empaqueta un bloque de código bajo un nombre para reutilizarlo múltiples veces sin reescribirlo.
- **Return:** Se usa *dentro* de una función para devolver un resultado final al programa principal.
- **Class:** Crea "moldes" o planos para la Programación Orientada a Objetos. Permite heredar características de otras clases.
- **Try/Except:** Red de seguridad. Intenta ejecutar un código y si ocurre un error, ejecuta un plan de emergencia para evitar que el programa colapse. EduBot Forest Studio lo configura automáticamente como `except Exception as e:`, lo que permite atrapar el error exacto y guardarlo en la variable `e` para saber exactamente qué falló.
- **Import:** Permite traer librerías o herramientas externas de Python a tu proyecto.

Ejemplo Práctico: Clase Básica (Evitando Errores)

Para crear una clase sin que Python arroje un error por estar vacía:

1. Arrastra un bloque **Class** y dale un nombre en el campo Name (ej. Perro).
2. Conecta a su salida un bloque de **Control** configurado en **Pass**.



Ejemplo Práctico: Atrapando Errores (División por Cero)

En matemáticas, dividir un número entre cero es imposible y hace que cualquier programa colapse. Vamos a usar la red de seguridad para evitarlo:

1. Coloca un bloque **Try** en tu lienzo.
2. En la ruta del **Try** (lo que intentaremos hacer), conéctale un bloque **Math** configurado para dividir (/). Coloca un 10 de un lado y un 0 del otro y guarda el resultado en una variable llamada resultado.
3. Ahora, en la ruta del **Except** (el plan de emergencia), conecta un bloque **Print**.
4. Configura ese Print para que muestre el texto *"No se puede dividir entre 0!"*.

Traducción de EduBot (Manejo de Errores)

```
class Perro:
```

```
    pass
```

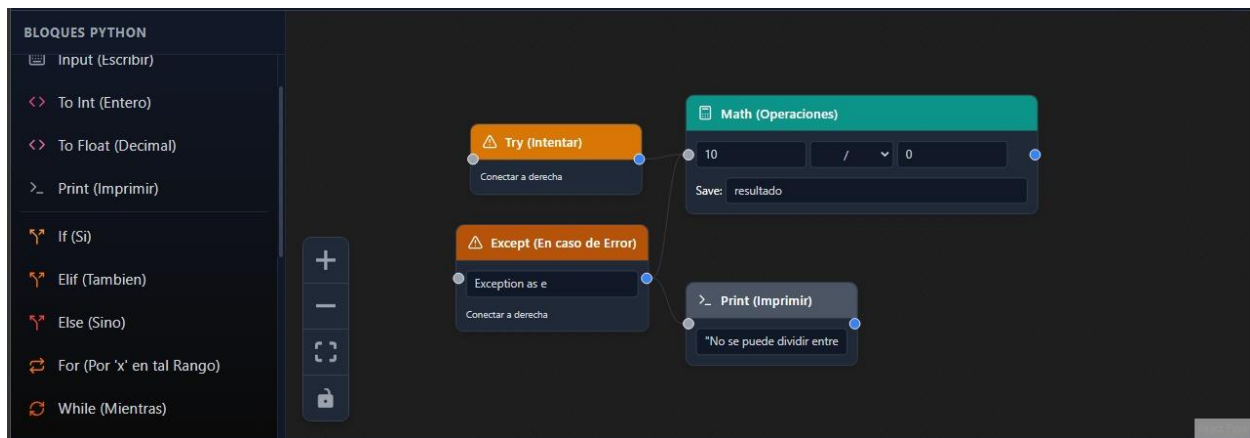
```
try:
```

```
    resultado = 10 / 0
```

```
except Exception as e:
```

```
    print("No se puede dividir entre 0!")
```

```
    print(f"El error técnico fue: {e}")
```



Herramientas Especiales del Entorno

EduBot Forest Studio cuenta con características de fondo para proteger tu equipo y evaluar el aprendizaje.

- **El Sandbox (La Zona Segura):** Una vez traducido el código, EduBot no lo ejecuta directamente en tu computadora, sino que lo mete en un *Sandbox* (Caja de Arena), una zona aislada y segura. Si el Sandbox detecta que el código tarda demasiado (como en un bucle infinito), lo detiene automáticamente (Timeout) y te devuelve un mensaje de error amigable para que corrijas tu lógica sin dañar el equipo.
- **El Interceptor de Notas (Validador de Retos):** Es un nodo tipo "Caja Negra" para evaluación. Este nodo comprueba si tu variable final coincide con la respuesta que el profesor esperaba. Si acertaste, envía silenciosamente un mensaje (EDUBOT_VAL_PASS) a la base de datos local de tu profesor para la evaluación automática.

Usen EduBot con sabiduría, después de todo, este es el preview oficial de la aplicación de escritorio. Pueden haber actualizaciones, con más cosas y más documentación al respecto, no se queden solo con lo que ven ahora, experimenten con la app y vean qué pueden hacer con ella con la lógica de programación pura.

¡EduBot Team está alegre de darles a conocer y probar la primera versión oficial de **EduBot Forest Studio**! ¡Aprovéchenlo!